

Discovery Agents for Real-Time Analytics: Toward Proactive Insight Systems

Gaetano Rossiello

IBM

New York City, New York, USA

Dharmashankar Subramanian

IBM

Yorktown Heights, New York, USA

Abstract

Modern analytics systems are fundamentally reactive, requiring users to define queries over increasingly complex and continuously evolving data. In real-time streaming environments, this paradigm breaks down, as the space of potential insights becomes too large to enumerate manually. We present a multi-agent architecture for autonomous insight discovery over real-time data streams. The system implements a continuous discovery loop in which agents generate hypotheses, compile them into executable analytics, validate generated artifacts, and produce visualizations and deployable applications. The architecture leverages Apache Kafka for event-driven coordination, Apache Flink for stream processing, and large language models to implement specialized agents. A key contribution is a contract-driven design based on typed intermediate artifacts, enabling modularity, observability, lineage, and safer execution of dynamically generated analytics. Through use cases in retail, finance, and public data, we show how this architecture supports a shift from query-driven analytics to proactive, discovery-driven systems.

1 Introduction

Data analytics systems have evolved from manual SQL workflows to dashboards and, more recently, LLM-powered copilots. Despite these advances, the dominant interaction model remains largely unchanged: analytics is still *query-driven*. Users formulate questions, and systems translate them into computations, visualizations, or reports. This model assumes that users know what to ask. In modern data environments, this assumption increasingly fails. Organizations operate over high-dimensional, heterogeneous, and continuously updated data sources. In particular, real-time streaming systems such as Apache Kafka [6] introduce a dynamic setting in which new patterns may emerge continuously, while the space of possible analytical questions grows combinatorially. As a result, many insights remain undiscovered not because they are difficult to compute, but because no analyst explicitly formulated the corresponding hypothesis.

Recent work on LLM-powered data agents has begun to automate parts of the analytics lifecycle, including data preparation, code generation, visualization, insight generation, and reporting [5, 18, 21, 25]. Existing systems demonstrate the potential of agents for business intelligence, dashboard generation, insight management, heterogeneous data analysis, and multi-agent reasoning [1, 9, 14, 20, 22, 24]. Benchmarks for data-driven discovery and analytics agents further show that multi-step insight generation, hypothesis formulation, and robust execution are becoming central evaluation problems [7, 15, 19]. However, most existing work primarily targets batch datasets, user-specified goals, conversational analysis, or dashboard generation. Less attention has been given

to how autonomous discovery agents can operate continuously over real-time streams, coordinate through production data infrastructure, and expose typed intermediate artifacts for observability, validation, and deployment.

We propose a shift from *task-driven analytics* to *discovery-driven analytics*, where analytical objectives are generated, validated, and refined by the system itself. Our approach decomposes the analytics lifecycle into specialized agents that transform data into hypotheses, executable analytics, validation reports, visualization specifications, and deployable applications. More broadly, we study the following operational model: *analytics as continuous autonomous discovery over real-time data streams*.

This paper makes three contributions. First, we formulate autonomous insight discovery as a continuous agentic workload over streaming data. Second, we propose a contract-driven architecture in which agents exchange typed artifacts for hypotheses, analytic plans, generated code, validation reports, visualization specifications, and deployment manifests. Third, we discuss representative use cases in retail, finance, and public data, highlighting early lessons for observability, validation, and human oversight in agent-driven analytics systems.

2 Related Work

LLM-powered data agents are emerging as a general abstraction for automating data management, preparation, analysis, visualization, and reporting. Recent surveys characterize these systems as tool-using agents that combine planning, retrieval, code generation, execution, and reflection across the data lifecycle [5, 18, 21, 25], while recent systems demonstrate their potential for heterogeneous data analysis, business intelligence, skill selection, external knowledge retrieval, and multi-agent reasoning [1, 9, 14, 20]. Related work on automated visualization, insight management, and data-to-dashboard generation shows how LLMs can produce charts, visual explanations, dashboard specifications, and navigable insight spaces from data and user intent [4, 8, 22–24]. In parallel, benchmarks and discovery-oriented systems increasingly frame insight generation as the target capability, evaluating agents on multi-step discovery, open-ended data analysis, business insight generation, heterogeneous data sources, and scientific hypothesis exploration [2, 7, 13, 15–17, 19]. Finally, work at the intersection of LLMs and databases highlights persistent challenges around trust, correctness, efficiency, and safe execution [10, 12]. Our work builds on these directions but differs in focus: rather than targeting static datasets, user-specified goals, reports, or interactive dashboards, we propose a streaming, contract-driven architecture in which agents continuously generate hypotheses, compile analytics, validate artifacts, produce visualizations, and package the result into deployable applications.

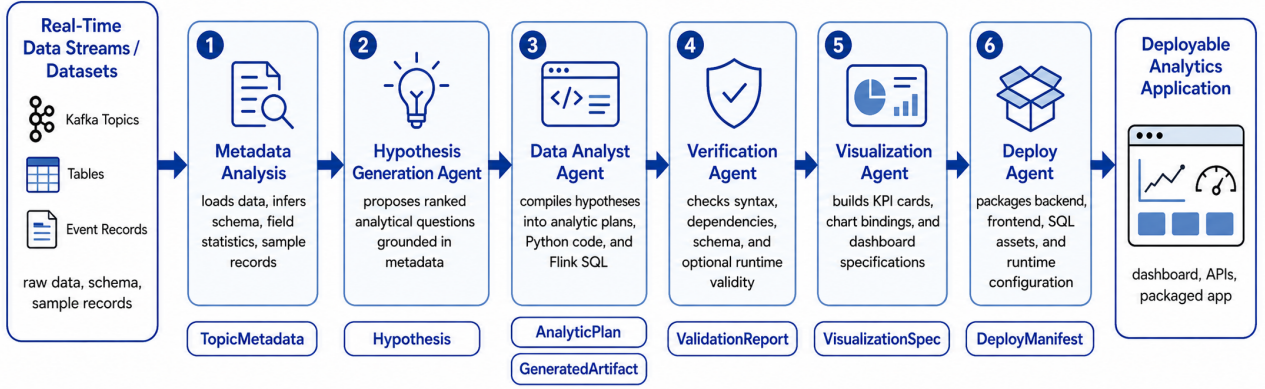


Figure 1: Contract-driven discovery-agent pipeline for real-time analytics. Real-time data streams or datasets are first converted into structured metadata, then transformed by specialized agents into ranked hypotheses, analytic plans, executable Python and FlinkSQL [3] artifacts, validation reports, visualization specifications, and finally a deployable analytics application. Each stage produces typed intermediate artifacts that support lineage, observability, verification, and human oversight.

3 Architecture: Discovery Agents over Streaming Data

The proposed system implements a multi-agent architecture for transforming real-time data streams into deployable analytics applications. At a high level, the system follows the pipeline (see Figure 1):

Data → *Metadata* → *Hypotheses* → *Analytics* → *Validation* → *Visualization* → *Application*

Unlike traditional data pipelines, where transformations are specified in advance, the computational workflow is constructed dynamically at runtime. Raw data is first converted into a semantic representation, *TopicMetadata*, capturing schema information, field statistics, sample records, and contextual metadata. This representation is then used to generate analytical hypotheses, which are compiled into executable artifacts, validated, visualized, and finally packaged into deployable applications.

A central design principle is the use of *typed artifact contracts*. Each agent consumes and produces explicit intermediate representations such as *Hypothesis*, *AnalyticPlan*, *GeneratedArtifact*, *ValidationReport*, *VisualizationSpec*, and *DeployManifest*. These contracts make the discovery process modular, inspectable, and reproducible.

The architecture supports both batch and streaming execution. In streaming deployments, Kafka acts as the communication backbone between agents, enabling asynchronous coordination and parallel exploration of hypotheses. Flink supports execution of generated SQL artifacts over continuous streams and provides operational visibility into the analytical pipeline. Together, these components allow the system to operate as a continuous discovery loop, where new data can trigger new hypotheses and updated analytics without manual query authoring.

4 Agent Semantics and Artifact Contracts

The system decomposes the analytics lifecycle into specialized agents, each responsible for a semantic transformation over typed

artifacts. This design avoids monolithic LLM workflows and makes the discovery process observable at each stage.

4.1 Hypothesis Generation Agent

The Hypothesis Generation Agent initiates the discovery process by proposing analytical questions from data characteristics. It consumes *TopicMetadata*, including schema definitions, field statistics, sample records, and optional domain annotations.

The agent is grounded in the taxonomy of data-analysis questions proposed by Leek and Peng [11], which distinguishes descriptive, exploratory, inferential, predictive, causal, and mechanistic questions. This taxonomy is useful because different question types imply different analytical assumptions and methods: descriptive questions summarize observed data, exploratory questions search for patterns, inferential questions ask whether patterns generalize, predictive questions estimate future or unseen outcomes, causal questions ask about interventions, and mechanistic questions seek to explain underlying processes.

Before generating hypotheses, the agent categorizes fields into analytical roles, such as numerical, categorical, temporal, or textual attributes. This grounding step constrains generation to questions that can be expressed using available data operations. For example, numerical fields may support descriptive summaries or anomaly detection, categorical fields may support exploratory group comparisons, and temporal fields may support predictive trend analysis.

The output is a set of structured *Hypothesis* objects:

```

{
  "category": "descriptive",
  "question": "...",
  "rationale": "...",
  "expected_insight": "...",
  "priority": 8
}
  
```

Each hypothesis includes a category, rationale, expected insight, and priority score. This structure allows downstream agents to

select appropriate analytical strategies and focus computational resources on higher-value questions.

4.2 Data Analyst Agent

The Data Analyst Agent translates hypotheses into executable analytics. Given a `Hypothesis` and the corresponding `TopicMetadata`, it produces an `AnalyticPlan` and one or more `GeneratedArtifact` objects.

A key design choice is dual artifact generation. Python artifacts provide flexible batch execution and support complex analytical logic, while FlinkSQL artifacts express equivalent computations for continuous stream processing. This allows the same analytical intent to be evaluated both offline and over live data streams.

The agent first constructs an analytic plan that identifies the required fields, the intended computation, the expected output schema, and the assumptions needed to answer the hypothesis. This planning step separates *what* should be computed from *how* it is implemented, making the generated code easier to validate, debug, and regenerate when failures occur.

Generated Python artifacts follow a strict runtime contract:

```
def analyze(data_records):  
    return {"results": [...]}
```

The output structure is standardized so that downstream validation, visualization, and deployment agents can consume results without relying on ad hoc parsing. Conceptually, the Data Analyst Agent acts as a compiler from analytical intent to executable programs.

This compiler-like role is important because different hypotheses require different analytical operators. Descriptive hypotheses may compile into aggregations and summaries, exploratory hypotheses into group comparisons or correlations, predictive hypotheses into trend or forecasting logic, and temporal hypotheses into windowed computations. By making these choices explicit in the `AnalyticPlan`, the system preserves a traceable link between the original question, the selected analytical method, and the generated executable artifact.

4.3 Verification Agent

The Verification Agent acts as a quality gate for generated artifacts. Since LLM-generated code may contain syntax errors, invalid assumptions, unsafe imports, or schema mismatches, validation is treated as a first-class stage rather than an optional post-processing step. This stage turns artifact correctness into an explicit, inspectable object, rather than leaving reliability as an implicit property of the generated code.

The agent performs syntax checks, dependency checks, schema validation, and, when sample data is available, runtime validation. The output is a structured `ValidationReport`:

```
{  
  "status": "VALIDATED",  
  "syntax_check": true,  
  "schema_check": true,  
  "runtime_check": false,  
  "issues": []  
}
```

Validation issues are classified as errors, warnings, or informational messages. Errors prevent artifacts from progressing, while warnings allow execution with caveats. Rejected artifacts can trigger regeneration with validation feedback, enabling iterative refinement.

4.4 Visualization Agent

The Visualization Agent converts validated analytical outputs into dashboard specifications. Rather than requiring manual chart design, the agent inspects output schemas and, when possible, sample execution results to infer appropriate visual encodings. It maps the semantic shape of the result—for example categorical distributions, temporal trends, scalar summaries, or ranked lists—to dashboard elements such as bar charts, line charts, KPI cards, tables, and summary views.

The output is a `VisualizationSpec` describing charts, KPI cards, data bindings, and layout:

```
{  
  "charts": [  
    {  
      "type": "bar",  
      "x": "category",  
      "y": "count"  
    }  
  ],  
  "kpis": [...]  
}
```

This specification separates analytical computation from presentation. It also enables multiple frontends to render the same discovered insight while preserving a traceable connection to the originating hypothesis and generated artifact.

4.5 Deploy Agent

The Deploy Agent materializes the validated analytics and visualization specifications into a runnable application. It packages generated code, dashboard configuration, backend services, frontend components, and deployment metadata. It also preserves the connection between runtime assets and upstream artifacts, so that deployed dashboards can be traced back to the hypotheses, code, validation reports, and visualization specifications that produced them. The output is a `DeployManifest` that records generated files, configuration parameters, artifact lineage, and deployment instructions. This manifest serves as the final contract between the discovery pipeline and the runtime environment. In this way, the system does not merely generate insights; it packages them into inspectable applications that can be executed, reviewed, and extended by users.

4.6 Contract-Driven Coordination

Typed artifact contracts provide the coordination layer across agents. Each transformation is explicit:

```
TopicMetadata → Hypothesis → AnalyticPlan →  
GeneratedArtifact → ValidationReport → VisualizationSpec →  
DeployManifest
```

This structure enables lineage tracking across the discovery process. Hypotheses reference their source topics, generated artifacts

reference the hypotheses they implement, validation reports reference the artifacts they evaluate, and deployment manifests reference all upstream objects.

In streaming deployments, these artifacts can be exchanged through Kafka topics, allowing multiple hypotheses to be explored in parallel. This creates a natural execution model for agentic analytics: agents operate asynchronously, artifacts provide the shared state, and validation reports control progression through the pipeline. The same contract structure also creates intervention points for human oversight. Users can review hypotheses before execution, inspect generated code before deployment, examine validation reports, or adjust visualization specifications. This balances automation with control and makes the system suitable for production-oriented analytics settings.

5 Use Cases and Early Lessons

Given the exploratory nature of the system, we evaluate it through representative use cases rather than a formal benchmark. The goal is to characterize the types of discovery workflows enabled by the architecture and identify early design lessons.

5.1 Retail Analytics

In retail analytics, the system processes transaction streams containing product, category, and customer interaction data. The Hypothesis Generation Agent identifies candidate patterns such as category dominance, temporal purchasing trends, basket composition, and anomalies in sales distribution. The system then generates aggregation queries, time-series analyses, and dashboards highlighting indicators such as top-selling categories, changes in product mix, or distribution shifts. This illustrates how proactive hypothesis generation can reduce the manual effort required to design exploratory dashboards.

5.2 Financial Monitoring

In financial monitoring, the system operates over transaction or market-event streams and proposes hypotheses related to volume spikes, unusual activity distributions, deviations from expected behavior, or temporal concentration of events. Generated analytics can surface candidate anomalies and summarize them through interpretable dashboards. Continuous hypothesis generation is particularly useful in this setting because relevant patterns may evolve rapidly and may not be known in advance.

5.3 Governance and Public Data

In public data scenarios, such as NYC Open Data, the system operates over heterogeneous datasets related to public services, infrastructure, complaints, or demographic indicators. It can propose hypotheses about temporal trends, geographic differences, or relationships between service requests and neighborhoods. The resulting dashboards can help analysts and policymakers explore datasets without first defining a fixed set of queries. This use case highlights the value of autonomous discovery in broad, heterogeneous data environments where the relevant analytical questions may not be obvious upfront.

5.4 Cross-Domain Lessons

Across these scenarios, three lessons emerge. First, hypothesis generation must be grounded in metadata and field statistics to avoid irrelevant or non-executable questions. Second, validation must be treated as a first-class stage, since generated code and queries may fail syntactically, semantically, or operationally. Third, observability over intermediate artifacts is essential: users and operators need to inspect not only final dashboards, but also the hypotheses, plans, code, and validation reports that produced them. These lessons suggest that agentic analytics systems require more than LLM prompting. They require system-level support for contracts, lineage, verification, and human oversight.

6 Position and Open Challenges

We argue that proactive insight discovery should become a first-class workload for agent-first data systems. The proposed architecture shifts analytics from a query-driven process, where goals are specified externally, to a discovery-driven process, where agents generate and prioritize analytical objectives directly from data. This elevates autonomy from execution to *problem formulation*. Instead of acting only as a query processor, the system operates as a continuous discovery engine. Analytical objectives emerge from the interaction between data, metadata, validation feedback, and specialized agents. The contract-driven design is a key enabler of this transition. Typed artifacts make LLM-based generation more controllable by exposing intermediate representations that can be validated, logged, inspected, and replayed. This is especially important in streaming environments, where agent actions may be triggered continuously and must remain auditable.

Several open challenges remain. First, hypothesis quality is difficult to evaluate: a valid hypothesis may still be uninteresting, redundant, or misleading. Second, generated analytics must be checked not only for syntax, but also for statistical validity and operational safety. Third, continuous discovery may produce many candidate insights, requiring ranking, deduplication, and human-in-the-loop review. Finally, deployment raises governance questions around provenance, access control, and accountability for system-generated conclusions. These challenges suggest a broader research agenda for agentic data systems: building infrastructure that supports not only faster query answering, but also reliable, observable, and steerable autonomous exploration.

7 Conclusion

We presented a multi-agent architecture for autonomous discovery over real-time data streams. By combining Kafka-based coordination, Flink-based stream processing, and LLM-powered agents with typed artifact contracts, the system transforms data into hypotheses, analytics, validation reports, visualizations, and deployable applications. The central contribution is a shift from automating analysis to automating discovery. Rather than accelerating only user-specified workflows, the proposed architecture treats insight generation itself as a continuous, agentic process embedded within the data infrastructure.

References

- [1] Amirhossein Abaskohi, Amrutha Varshini Ramesh, Shailesh Nanisetty, Chirag Goel, et al. 2025. AgentAda: Skill-Adaptive Data Analytics for Tailored Insight Discovery. *arXiv preprint arXiv:2504.07421* (2025).
- [2] Dhruv Agarwal, Bodhisattwa Prasad Majumder, Reece Adamson, Megha Chakraborty, et al. 2025. AutoDiscovery: Open-ended Scientific Discovery via Bayesian Surprise. *arXiv preprint arXiv:2507.00310* (2025).
- [3] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering* 38, 4 (2015).
- [4] Zichen Chen, Jiefeng Chen, Serkan O. Arik, Misha Sra, Tomas Pfister, and Jinsung Yoon. 2025. CoDA: Agentic Systems for Collaborative Data Visualization. *arXiv preprint arXiv:2510.03194* (2025).
- [5] Yanjie Fu, Dongjie Wang, Wangyang Ying, Xiangliang Zhang, Huan Liu, and Jian Pei. 2025. Autonomous Data Agents: A New Opportunity for Smart Data. *arXiv preprint arXiv:2510.03194* (2025).
- [6] Nishant Garg. 2013. *Apache kafka*. Packt Publishing.
- [7] Ken Gu, Ruoxi Shang, Ruien Jiang, Keying Kuang, et al. 2024. BLADE: Benchmarking Language Model Agents for Data-Driven Science. In *Findings of the Association for Computational Linguistics: EMNLP*.
- [8] Enamul Hoque and Mohammed Saidul Islam. 2025. Natural Language Generation for Visualizations: State of the Art, Challenges and Future Directions. *Computer Graphics Forum* (2025).
- [9] Jie Jiang, Haining Xie, Siqi Shen, Yu Shen, Zihan Zhang, Meng Lei, Yifeng Zheng, Yang Li, Chunyou Li, Danqing Huang, et al. 2025. SiriusBI: A Comprehensive LLM-Powered Solution for Data Analytics in Business Intelligence. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4860–4873.
- [10] Kyoungmin Kim and Anastasia Ailamaki. 2024. Trustworthy and Efficient LLMs Meet Databases. *arXiv preprint arXiv:2412.18022* (2024).
- [11] Jeffery T Leek and Roger D Peng. 2015. What is the question? *Science* 347, 6228 (2015), 1314–1315.
- [12] Guoliang Li et al. 2024. LLM for Data Management. *arXiv preprint* (2024).
- [13] Shicheng Liu, Yucheng Jiang, Sajid Farook, Camila Nicollier Sanchez, David Fernando Castro Pena, and Monica S. Lam. 2026. DataSTORM: Deep Research on Large-Scale Databases using Exploratory Data Analysis and Data Storytelling. *arXiv preprint arXiv:2604.06474* (2026).
- [14] Xiaochuan Liu, Yuanfeng Song, Xiaoming Yin, and Xing Chen. 2025. DataSage: Multi-agent Collaboration for Insight Discovery with External Knowledge Retrieval, Multi-role Debating, and Multi-path Reasoning. *arXiv preprint arXiv:2511.14299* (2025).
- [15] Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, et al. 2024. DiscoveryBench: Towards Data-Driven Discovery with Large Language Models. In *International Conference on Learning Representations*.
- [16] Bodhisattwa Prasad Majumder, Harshit Surana, Sanchaita Hazra, Ashish Sabharwal, and Peter Clark. 2024. Data-driven Discovery with Large Generative Models. In *International Conference on Machine Learning*.
- [17] Ludovico Mitchener, Angela Yiu, Benjamin Chang, Mathieu Bourdenx, et al. 2025. Kosmos: An AI Scientist for Autonomous Discovery. *arXiv preprint arXiv:2511.02824* (2025).
- [18] Mizanur Rahman, Amran Bhuiyan, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Ridwan Mahbub, Ahmed Masry, Shafiq Joty, and Enamul Hoque. 2025. LLM-Based Data Science Agents: A Survey of Capabilities, Challenges, and Future Directions. *arXiv preprint arXiv:2510.04023* (2025).
- [19] Gaurav Sahu, Abhay Puri, Juan Rodriguez, Amirhossein Abaskohi, et al. 2024. InsightBench: Evaluating Business Analytics Agents Through Multi-Step Insight Generation. *arXiv preprint arXiv:2407.06423* (2024).
- [20] Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. 2025. AgenticData: An Agentic Data Analytics System for Heterogeneous Data. *arXiv preprint arXiv:2508.05002* (2025).
- [21] Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2024. A Survey on Large Language Model-based Agents for Statistics and Data Science. *The American Statistician* (2024).
- [22] Luoxuan Weng, Xingbo Wang, Junyu Lu, Yingchaojie Feng, Yihan Liu, Haozhe Feng, Danqing Huang, and Wei Chen. 2025. InsightLens: Augmenting LLM-Powered Data Analysis with Interactive Insight Management and Navigation. *IEEE Transactions on Visualization and Computer Graphics* (2025).
- [23] Yang Wu, Yao Wan, Hongyu Zhang, Yulei Sui, Wucui Wei, Wei Zhao, Guandong Xu, and Hai Jin. 2024. Automated Data Visualization from Natural Language via Large Language Models: An Exploratory Study. *Proceedings of the ACM on Management of Data* (2024).
- [24] Ran Zhang and Mohannad Elhamod. 2025. Data-to-Dashboard: Multi-Agent LLM Framework for Insightful Visualization in Enterprise Analytics. *arXiv preprint arXiv:2505.23695* (2025).
- [25] Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, et al. 2025. A Survey of Data Agents: Emerging Paradigm or Overstated Hype? *arXiv preprint arXiv:2510.23587* (2025).